

# Lógica Computacional



Prof. Esp. Wesley Neves

# 13



## AULA 13 – Introdução a Python





# Conteúdo

## Python Essentials 1: Módulo 2

**Tipos de dados, variáveis, operações básicas de input-output, operadores básicos**

Neste módulo, aprenderá:

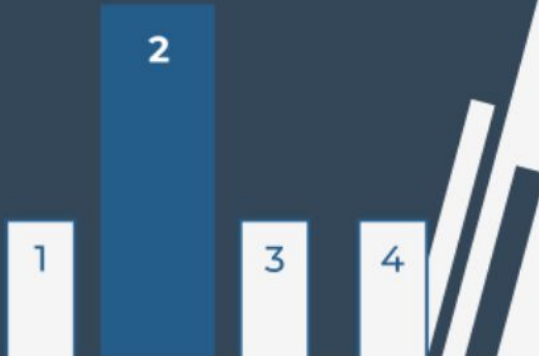
- como escrever e executar programas simples de Python;
- o que são literais, operadores, e expressões Python;
- o que são as variáveis e quais as regras que as regem;
- como realizar operações básicas de input e output.

# Introdução a Python

## Module 2:

Data types, variables,  
basic input-output operations,  
basic operators

Python Essentials 1



# Introdução a Python

## Hello, World!

É tempo de começar a escrever algum **código Python real e funcional**. Vai ser muito simples por enquanto.

Como vamos mostrar-lhe alguns conceitos e termos fundamentais, estes snippets de código não serão sérios ou complexos.

Execute o código na janela do editor à direita. Se tudo correr bem aqui, verá a **linha de texto** na janela da consola.

# Introdução a Python



```
1 print("Hello, World!")  
2
```

Como pode ver, o primeiro programa consiste nas seguintes partes:

- a palavra `print` ;
- um parêntesis de abertura;
- umas aspas;
- uma linha de texto: `Hello, World!` ;
- outras aspas;
- um parêntesis de fecho.

# Introdução a função

## A função `print()`

Veja a linha de código abaixo:

```
print("Hello, World!")
```

A palavra **print** que se pode ver aqui é um **nome de função**. Isso não significa que, onde quer que a palavra apareça, é sempre um nome de função. O significado da palavra vem do contexto em que a palavra foi usada.

Uma função (neste contexto) é uma parte separada do código do computador capaz de:

- **causar um qualquer efeito** (por exemplo, enviar texto para o terminal, criar um ficheiro, desenhar uma imagem, reproduzir um som, etc.); isto é algo completamente inédito no mundo da matemática;
- **avaliar um valor** (por exemplo, a raiz quadrada de um valor ou o comprimento de um dado texto) e **devolvê-lo como o resultado da função**; é isto que faz as funções Python serem os parentes dos conceitos matemáticos.

Além disso, muitas das funções Python podem fazer as duas coisas acima juntamente.

# Introdução a função

De onde vêm as funções?

- Podem vir **do próprio Python**; a função `print` é uma deste tipo; tal função é um valor acrescentado recebido juntamente com o Python e o seu ambiente **(é incorporada)** não é necessário fazer nada de especial (por exemplo, perguntar a alguém por qualquer coisa) se quiser fazer uso dela;
- podem ser provenientes de um ou mais **dos add-ons de Python chamados módulos** alguns dos módulos vêm com Python, outros podem requerer instalação separada - seja qual for o caso, todos eles precisam de estar explicitamente ligados ao seu código (mostrar-lhe-emos como fazê-lo em breve);
- pode **escrevê-los você mesmo**, colocando tantas funções quantas quiser e precisar dentro do seu programa para o tornar mais simples, mais claro e mais elegante.

# Introdução a função

Como dissemos antes, uma função pode ter:

- um **efeito**;
- um **resultado**.

Apesar do número de argumentos necessários/fornecidos, as funções Python exigem fortemente a presença de **um par de parêntesis** - de abertura e de fecho, respetivamente.

Nota: para distinguir palavras comuns de nomes de funções, coloque **um par de parêntesis vazios** após os seus nomes, mesmo que a função correspondente queira um ou mais argumentos. Esta é uma convenção padrão.

# Introdução a função

## A função `print()`

O nome da função (***print*** neste caso) juntamente com os parêntesis e o(s) argumento(s), formam a **invocação da função**.

Discutiremos isto com mais profundidade em breve, mas devemos dar-lhe umas luzes de momento.

```
print("Hello, World!")
```

O que acontece quando o Python encontra uma invocação como esta abaixo?

```
function_name(argument)
```

# Introdução a função

```
1.funcaoSomarnumero.py U X
PythonCisco > 2.Modulo2 > 1.funcaoSomarnumero.py > ...
1
2
3
4 def SomarDoisNumeros(numero1,numero2):
5     numero1 + numero2
6
7
8
9 SomarDoisNumeros(40,20)
10
11
```

# Introdução a função

```
2.funcaoSomarnumero.py U X
PythonCisco > 1.Modulo1 > 2.funcaoSomarnumero.py > ...
1
2
3 def SomarDoisNumeros(numero1,numero2):
4     numero1 + numero2
5
6
7 print(SomarDoisNumeros(50,50))
```

```
PS D:\Wesley\repos\AprendendoPython> &
None
```

Nesse caso aqui a função print retorna None ou seja “Nada”

por que a função não retorna nada , ela apenas executa.

# Introdução a função

```
PythonCisco > 1.Modulo1 > 2.funcaoSomarnumero.py > ...
```

```
1  
2  
3 def SomarDoisNumeros(numero1,numero2):  
4     return numero1 + numero2  
5  
6  
7 print(SomarDoisNumeros(50,50))
```

Nesse caso aqui a função print retorna 100 por que a função retorna o resultado do cálculo



LAB

## Tempo estimado

5-10 minutos

## Nível de dificuldade

Muito fácil

## Objetivos

- familiarizar-se com a função `print()` e as suas capacidades de formatação;
- experimentar com código Python.

## Cenário

O comando `print()`, que é uma das diretivas mais fáceis em Python, imprime simplesmente uma linha para o ecrã.

No seu primeiro lab:

- use a função `print()` para imprimir a linha `Hello, Python!` para o ecrã. Use aspas duplas à volta da string;
- tendo feito isso, utilize a função `print()` novamente, mas desta vez imprima o seu primeiro nome;
- remova as aspas duplas e execute o seu código. Veja a reação do Python. Que tipo de erro é lançado?
- depois, retire os parêntesis, volte a colocar as aspas duplas, e execute novamente o seu código. Que tipo de erro é lançado desta vez?
- experimente o máximo que puder. Altere aspas duplas com aspas simples, use várias funções `print()` na mesma linha e, em seguida, em linhas diferentes. Veja o que acontece.

Três questões importantes têm de ser respondidas assim que possível:

1. Qual é o efeito que a função `print()` causa?

2. Que argumentos `print()` espera?

3. Que valor é devolvido pela função `print()` ?



## A função `print()` - instruções

Mudámos um pouco o exemplo - acrescentámos uma invocação de função **vazia** `print()`. Chamamos-lhe vazia porque não apresentámos quaisquer argumentos para a função.

Pode vê-lo na janela do editor. Execute o código.

O que acontece?

Se tudo correr bem, deverá ver algo como isto:

```
The itsy bitsy spider climbed up the waterspout.  
  
Down came the rain and washed the spider out.
```

output

Como pode ver, a invocação vazia `print()` não é tão vazia como se poderia esperar - produz uma linha vazia, ou (esta interpretação também é correta) o seu output é apenas uma newline.

Esta não é a única forma de produzir uma **newline** na consola de output. Vamos agora mostrar-lhe outra forma.



# LAB

```
▶ [Icons: Copy, Paste, Share, Download, Upload, Refresh]
1 print("The itsy bitsy spider climbed up the waterspout.")
2 print()
3 print("Down came the rain and washed the spider out.")
4
```

Console >\_

The itsy bitsy spider climbed up the waterspout.

Down came the rain and washed the spider out.

# os caracteres de escape e de newline

```
1 print("The itsy bitsy spider\nclimbed up the waterspout.")
2 print()
3 print("Down came the rain\nand washed the spider out.")
4 |
```

Há duas mudanças muito subtis - inserimos um estranho par de caracteres dentro da rima. Têm este aspeto:

```
\n.
```

Curiosamente, enquanto **se pode ver dois caracteres, o Python vê um.**

A barra invertida ( `\` ) tem um significado muito especial quando usado dentro de strings - a isto chama-se o **caratere de escape.**

# os caracteres de escape e de newline

```
1 print("The itasy bitsy spider\nclimbed up the waterspout.")
2 print()
3 print("Down came the rain\nand washed the spider out.")
4
```

Console >\_

The itasy bitsy spider  
climbed up the waterspout.

Down came the rain  
and washed the spider out.

A palavra *escape* deve ser entendida especificamente - significa que a série de caracteres na string escapa por um momento (um momento muito curto) para introduzir uma inclusão especial.

Por outras palavras, a barra invertida não significa nada em si, mas é apenas uma espécie de anúncio de que o próximo caractere após a barra invertida também tem um significado diferente.

A letra `n` colocada após a barra invertida vem da palavra *newline* (nova linha).

# os caracteres de escape e de newline

## Caracteres de escape disponíveis em Python

- `\n` - Nova linha. Move o cursor para o início da próxima linha.
- `\t` - Tabulação Horizontal. Move o cursor para a próxima parada de tabulação (tab).
- `\r` - Retorno de carro. Move o cursor para o início da linha atual; não avança para a próxima linha.
- `\b` - Backspace. Retrocede o cursor um caractere.
- `\a` - Alerta. Emite o som de sino do sistema.
- `\` - Barra invertida (Backslash).
- `\"` - Caractere de aspas duplas.
- `\'` - Caractere de aspas simples.

# forma posicional de passar os parâmetros

A forma como estamos a passar os argumentos para a função `print()` é a mais comum em Python, e é chamada a **forma posicional** (este nome vem do facto de o significado do argumento ser ditado pela sua posição, por exemplo, o segundo argumento será produzido após o primeiro, e não o contrário).

Execute o código e verifique se o output corresponde às suas previsões.



```
1 print("My name is", "Python.")  
2 print("Monty Python.")  
3
```



# argumentos de keyword

E agora é tempo de tentar algo mais difícil.

Se olhar cuidadosamente, verá que utilizámos o argumento `end`, mas a string atribuída a ele está vazia (não contém nenhum caractere).

O que vai acontecer agora? Execute o programa no editor para descobrir.

+ Como o argumento `end` foi definido para nada, a função `print()` também não produz nenhum output, uma vez que os seus argumentos posicionais foram esgotados.



```
1 print("My name is ", end="")
2 print("Monty Python.")
3
```

Console >\_

My name is Monty Python.

LAB

## Tempo estimado

5-15 minutos

## Nível de dificuldade

Fácil

## Objetivos

- experimentar com o código Python existente;
- descobrir e corrigir erros básicos de sintaxe;
- familiarizar-se com a função `print()` e os seus recursos de formatação.

```

1 print("  *")
2 print(" * *")
3 print(" *  *")
4 print(" *   *")
5 print("****")
6 print(" *   *")
7 print(" *   *")
8 print(" ****")
9
10
11 print(" " * 3, "*\n", " " * 1, "* *\n", " " * 1, "*   *")
12
13

```



1. A função `print()` é uma função **incorporada**. Imprime/faz output de uma mensagem especificada para a janela do ecrã/console.
2. As funções incorporadas, ao contrário das funções definidas pelo utilizador, estão sempre disponíveis e não têm de ser importadas. O Python 3.8 vem com 69 funções incorporadas. Pode encontrar a sua lista completa fornecida em ordem alfabética na [Biblioteca Padrão Python](https://docs.python.org/3/library/functions.html).

<https://docs.python.org/3/library/functions.html>

# Valores Literais e Não Literais

## Literais - os dados em si

Agora que tem um pouco de conhecimento de algumas das poderosas características oferecidas pela função `print()`, é hora de aprender sobre algumas novas questões, e um novo termo importante - o **literal**.

**Um literal são dados cujos valores são determinados pelo próprio literal.**

Como este é um conceito difícil de compreender, um bom exemplo pode ser útil.

Dê uma vista de olhos ao seguinte conjunto de dígitos:

Consegue adivinhar que valor representa? Claro que consegue - é *cento e vinte e três*.

Mas e quanto a isto:

Representa algum valor? Talvez. Pode ser o símbolo da velocidade da luz, por exemplo. Também pode ser a constante de integração. Ou mesmo o comprimento de uma hipotenusa no sentido de um teorema de Pitágoras. Existem muitas possibilidades.

# Valores Literais e Não Literais

## Literais - os dados em si

Começemos com uma simples experiência - veja o snippet no editor.

A primeira linha parece familiar. A segunda parece estar errada devido à visível falta de aspas.

Tente executá-lo.

Se tudo correu bem, deverá agora ver duas linhas idênticas.

O que aconteceu? O que significa isto?

Através deste exemplo, depara-se com dois tipos diferentes de literais:

- uma **string**, que já conhece,
- e um número **inteiro**, algo completamente novo.

```
1 print("2")
2 print(2)
3
```

Console >\_

```
2
2
```

```
1 print("2")
2 print(2)
3
4 print(type("2"))
5 print(type(2))
6
```

```
<class 'str'>
<class 'int'>
```

# Valores Literais e Não Literais

## Literais - os dados em si

Começemos com uma simples experiência - veja o snippet no editor.

A primeira linha parece familiar. A segunda parece estar errada devido à visível falta de aspas.

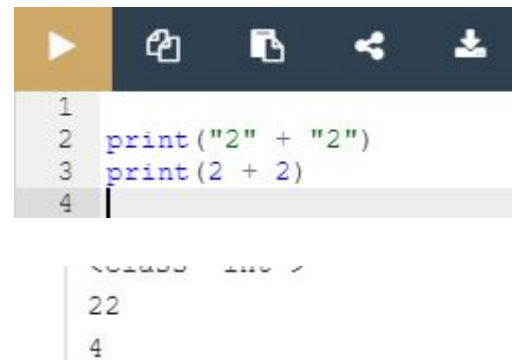
Tente executá-lo.

Se tudo correu bem, deverá agora ver duas linhas idênticas.

O que aconteceu? O que significa isto?

Através deste exemplo, depara-se com dois tipos diferentes de literais:

- uma **string**, que já conhece,
- e um número **inteiro**, algo completamente novo.



```
1  
2 print("2" + "2")  
3 print(2 + 2)  
4  
22  
4
```

# Inteiros

## Inteiros

Pode já saber um pouco sobre como os computadores efetuam cálculos sobre números. Talvez já tenha ouvido falar do **sistema binário**, e saiba que é o sistema que os computadores utilizam para armazenar números, e que eles podem realizar qualquer operação sobre eles.

Não vamos explorar aqui as complexidades dos sistemas de numeração posicional, mas vamos dizer que os números tratados pelos computadores modernos são de dois tipos:

- **inteiros**, isto é, aqueles que são desprovidos da parte fraccionada;
- e números **floating-point** (ou simplesmente **float**), que contêm (ou são capazes de conter) a parte fraccionada.



# Inteiros: números octais e hexadecimais

## Inteiros: números octais e hexadecimais

Há duas convenções adicionais em Python que são desconhecidas no mundo da matemática. A primeira permite-nos utilizar números numa representação **octal**.

Se um número inteiro for precedido por um `0o` ou `0O` prefixo (zero-o), ele será tratado como um valor octal. Isto significa que o número deve conter apenas dígitos retirados do intervalo [0..7].

`0o123` é um número **octal** com um valor (decimal) igual a `83`.

A classe `print()` faz a conversão automaticamente. Experimente isto:

```
print(0o123)
```



# Inteiros: números octais e hexadecimais

A segunda convenção permite-nos utilizar números **hexadecimais**. Estes números devem ser precedidos pelo prefixo `0x` ou `0X` (zero-x).

`0x123` é um número **hexadecimal** com um valor (decimal) igual a `291`. A função `print()` também pode gerir estes valores. Experimente isto:

```
print(0x123)
```

SECTION (33%)



```
1 print(0o123)
```

```
2
```

```
3 print(0x123)|
```



## Floats

Agora é altura de falar de outro tipo, que foi concebido para representar e armazenar os números que (como diria um matemático) têm uma **fração decimal não vazia**.

São os números que têm (ou podem ter) uma parte fracionada após o ponto decimal, e embora tal definição seja muito pobre, é certamente suficiente para o que desejamos discutir.

Sempre que utilizamos um termo como *dois e meio* ou *menos zero ponto quatro*, pensamos em números que o computador considera números de **floating-point**:

2.5

-0.4



# Floats

Como provavelmente pode imaginar, o valor de **zero ponto quatro** pode ser escrito em Python como:

```
0.4
```

Mas não se esqueça desta regra simples - pode omitir o zero quando é o único dígito em frente ou após o ponto decimal.

Em essência, pode escrever o valor `0.4` como:

```
.4
```



# Ints vs. floats

Como provavelmente pode imaginar, o valor de **zero ponto quatro** pode ser escrito em Python como:

```
0.4
```

Mas não se esqueça desta regra simples - pode omitir o zero quando é o único dígito em frente ou após o ponto decimal.

Em essência, pode escrever o valor `0.4` como:

```
.4
```



# Ints vs. floats

O ponto decimal é essencialmente importante no reconhecimento de números de floating-point em Python.

Veja estes dois números:

4

4.0

Pode pensar que eles são exatamente os mesmos, mas o Python vê-os de uma forma completamente diferente.

4 é um número **inteiro**, enquanto que 4.0 é um número **floating-point**.

O ponto é o que faz um float.



UNIDESC

# Ints vs. floats

Por outro lado, não são apenas os pontos que fazem um float. Também pode utilizar a letra `e`.

Quando quiser usar números muito grandes ou muito pequenos, pode usar **notação científica**.



# Ints vs. floats

Tome, por exemplo, a velocidade da luz, expressa em *metros por segundo*. Escrito diretamente, ficaria assim: 3000000000 .

Para evitar escrever tantos zeros, os livros de física utilizam uma forma abreviada, que provavelmente já viu:  $3 \times 10^8$  .

Lê-se: três vezes dez à potência de oito.

Em Python, o mesmo efeito é conseguido de uma forma ligeiramente diferente - veja:

3E8

A letra E (também pode utilizar a letra minúscula e - vem da palavra **expoente**) é um registo conciso da frase *vezes dez à potência de*.



# Codificação de floats

UNIDESC

## Codificação de floats

Vejamos como esta convenção é utilizada para registrar números que são muito pequenos (no sentido do seu valor absoluto, que está próximo de zero).

Uma constante física chamada *constante de Planck* (e denotada com um  $h$ ), de acordo com os manuais escolares, tem o valor de:  
 **$6.62607 \times 10^{-34}$** .

Se quiser utilizá-la num programa, deve escrevê-la desta forma:

```
6.62607E-34
```



UNIDESC

# Codificação de floats

Por exemplo, digamos que decidiu usar o seguinte float literal:

```
0.000000000000000000000001
```

Quando executa este literal através do Python:

```
print(0.000000000000000000000001)
```

este é o resultado:

```
1e-22
```

O Python escolhe sempre **a forma mais económica de apresentação do número**, e deve ter isto em consideração ao criar literais.



# Strings

UNIDESC

Strings são utilizadas quando é necessário processar texto (como nomes de todos os tipos, endereços, romances, etc.), não números.

Já sabe um pouco sobre elas. Por exemplo, que as **strings precisam de aspas** da mesma forma que floats precisam de pontos.

Esta é uma string muito típica: `"I am a string."`

No entanto, há um senão. O senão é como codificar uma aspa dentro de uma string que já está delimitada por aspas.



# Strings

UNIDESC

No entanto, há um senão. O senão é como codificar uma aspa dentro de uma string que já está delimitada por aspas.

Vamos supor que queremos imprimir uma mensagem muito simples dizendo:

```
I like "Monty Python"
```

Como fazemos isto sem gerar um erro? Existem duas soluções possíveis.

A primeira baseia-se no conceito que já conhecemos do **caratere de escape**, que como se deve lembrar é representado pela **barra invertida**. A barra invertida também pode escapar às aspas. Uma aspa precedida por uma barra invertida muda o seu significado - não é um delimitador, mas apenas uma aspa. Isto funcionará como pretendido:

```
print("I like \"Monty Python\"")
```



# Strings

No entanto, há um senão. O senão é como codificar uma aspa dentro de uma string que já está delimitada por aspas.

Vamos supor que queremos imprimir uma mensagem muito simples dizendo:

```
I like "Monty Python"
```

Como fazemos isto sem gerar um erro? Existem duas soluções possíveis.

Este exemplo também funcionará:

```
print('I like "Monty Python"')
```



# Strings

UNIDESC



```
1 print("I'm Monty Python")  
2  
3 print("I"m Monty Python)|
```

```
print("I"m Monty Python")  
      ^  
SyntaxError: invalid syntax
```



UNIDESC

# Valores Booleanos

Cada vez que pergunta ao Python se um número é maior que outro, a pergunta resulta na criação de alguns dados específicos - um valor **Booleano**.

O nome vem de George Boole (1815-1864), autor da obra fundamental, *As Leis do Pensamento*, que contém a definição de **álgebra Booleana** - uma parte da álgebra que faz uso de apenas dois valores distintos: `True` e `False`, denotado como `1` e `0`.



# Valores Booleanos

O Python, então, é um réptil **binário**.

Estes dois valores Booleanos têm denotações rigorosas em Python:

```
True  
False
```

Não se pode mudar nada - é preciso tomar estes símbolos tal como eles são, incluindo **case-sensitivity**.

Desafio: Qual será o output do seguinte snippet de código?

```
print(True > False)  
print(True < False)
```



UNIDESC

# Valores Booleanos

LAB

## Tempo estimado

5-10 minutos

## Nível de dificuldade

Fácil

## Objetivos

- familiarizar-se com a função `print()` e as suas capacidades de formatação;
- praticar codificação de strings;
- experimentar com código Python.



UNIDESC

# Valores Booleanos

## Cenário

Escreva um código de uma linha, utilizando a função `print()`, bem como os caracteres de newline e de escape, para corresponder ao resultado esperado emitido em três linhas.

## Output esperado

```
"I'm"  
""learning""  
""Python""
```